



Assignment No.5

OBJECT ORIENTED PROGRAMMING

Vigyat Manktala

1024150373

2034

- \
1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```
#include <iostream> using
namespace std;
```

```
class Person {
public:
string name;
int age;
```

```
    void getPersonData() {
cout << "Enter name: ";
cin >> name;    cout <<
"Enter age: ";    cin >>
age;
    }
```

```
    void displayPersonData() {    cout
<< "Name: " << name << endl;    cout
<< "Age: " << age << endl;
    }
};
```

```
class Student : public Person { public:
    int rollNo;
```

```
    void getStudentData() {
cout << "Enter roll number: ";
cin >> rollNo;
    }
```

```
    void displayStudentData() {    cout <<
"Roll Number: " << rollNo << endl;
    }
};
```

```
int main() {
```

```

    Person p;    cout << "Enter
Person Details\n";
p.getPersonData();    cout <<
"\nPerson Details:\n";
p.displayPersonData();

    cout << "\n-----\n";

    Student s;    cout << "Enter
Student Details\n";
    s.getPersonData();    // inherited function
    s.getStudentData();

    cout << "\nStudent Details:\n";
    s.displayPersonData(); // inherited function
    s.displayStudentData();

    return 0;
}

```

```

Output

Enter Person Details
Enter name: Saanvi
Enter age: 20

Person Details:
Name: Saanvi
Age: 20

-----

Enter Student Details
Enter name: Ram
Enter age: 45
Enter roll number: 12

Student Details:
Name: Ram
Age: 45
Roll Number: 12

=== Code Execution Successful ===

```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

```
#include <iostream> using
namespace std;

class Animal { protected:
    int legs;

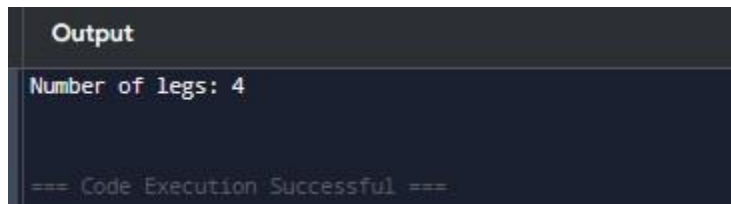
public:
    void setLegs(int l) {
        legs = l;
    }
};

class Dog : public Animal { public:
    void display() {      cout << "Number of legs:
" << legs << endl;  }
};

int main() {

    Dog d;
    d.setLegs(4);
    d.display();

    return 0;
}
```



The screenshot shows a dark-themed output window. At the top, the word "Output" is displayed in a light blue font. Below it, the text "Number of legs: 4" is shown in a light blue font. At the bottom, the text "=== Code Execution Successful ===" is displayed in a light blue font.

3. Most of the time we use public mode of inheritance, for example class Derived:

public Base{}; Try protected and private access modifiers to understand the difference of various modes of inheritance.

```
#include <iostream> using
namespace std;
```

```
class Base { public:
    int a;
```

```
protected:
    int b;
```

```
private:
    int c;
```

```
public:
    void setValues() {
        a = 10;
        b = 20;      c
        = 30;
    }

    void showBase() {      cout
    << "a = " << a << endl;      cout
    << "b = " << b << endl;      cout
    << "c = " << c << endl;
    }
};
```

```
class DerivedPublic : public Base { public:
    void display() {
        cout << a << endl;
        cout << b << endl;
    }
};
```

```
class DerivedProtected : protected Base { public:
    void display() {
        cout << a << endl;
        cout << b << endl;
    }
};
```

```
};
```

```
class DerivedPrivate : private Base { public:
```

```
    void display() {  
cout << a << endl;  
cout << b << endl;  
    }  
};
```

```
int main() {
```

```
    DerivedPublic obj1;  
obj1.setValues();  
obj1.display();  
    obj1.a = 50;    cout <<  
obj1.a << endl;
```

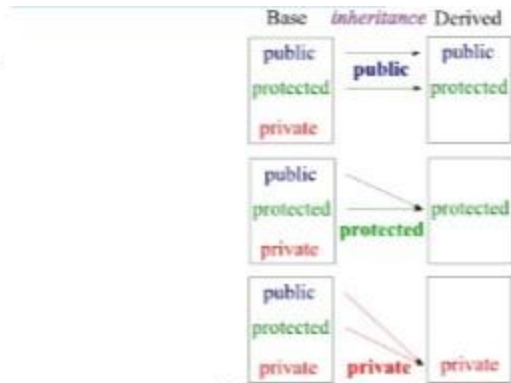
```
    DerivedProtected obj2;  
obj2.display();
```

```
    DerivedPrivate obj3;  
obj3.display();
```

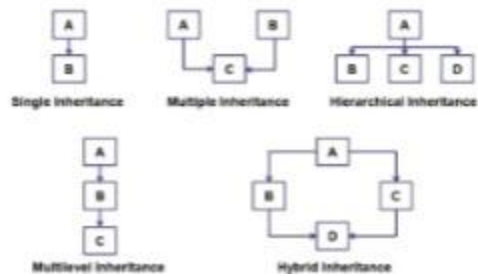
```
    return 0;  
}
```

```
Output  
10  
20  
50  
1651076199  
779647075  
32107  
1  
  
=== Code Execution Successful ===
```

4.



Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.



```
#include <iostream> using
namespace std;
```

```
class A1 { public:      void showA1() { cout << "Single
Inheritance A" << endl; }
};
```

```
class B1 : public A1 { public:  void showB1() { cout <<
"Single Inheritance B" << endl; }
};
```

```
class A2 { public:  void showA2() { cout << "Multiple
Inheritance A" << endl; }
};
```

```
class B2 { public:
void showB2() {
cout <<
"Multiple
```

Inheritance B"

```
<< endl; }
```

```
};
```

```
class C2 : public A2, public B2 { public:    void showC2() {  
cout << "Multiple Inheritance C" << endl; }
```

```
};
```

```
class A3 { public:    void showA3() { cout << "Hierarchical  
Inheritance A" << endl; }
```

```
};
```

```
class B3 : public A3 { public:    void showB3() { cout <<  
"Hierarchical Inheritance B" << endl; }
```

```
};
```

```
class C3 : public A3 { public:    void showC3() { cout <<  
"Hierarchical Inheritance C" << endl; }
```

```
};
```

```
class A4 { public:    void showA4() { cout << "Multilevel  
Inheritance A" << endl; }
```

```
};
```

```
class B4 : public A4 { public:    void showB4() { cout <<  
"Multilevel Inheritance B" << endl; }
```

```
};
```

```
class C4 : public B4 { public:    void showC4() { cout <<  
"Multilevel Inheritance C" << endl; }
```

```
};
```

```
class A5 { public:
```

```
void showA5() {
```

```
cout << "Hybrid
```

```
Inheritance A"
```

```
<< endl; }
```

```
};
```

```
class B5 : virtual public A5 { public:    void showB5() { cout  
<< "Hybrid Inheritance B" << endl; }
```



```
};
```

```
class C5 : virtual public A5 { public:    void showC5() { cout  
<< "Hybrid Inheritance C" << endl; }  
};
```

```
class D5 : public B5, public C5 { public:    void showD5() {  
cout << "Hybrid Inheritance D" << endl; }  
};
```

```
int main() {
```

```
    B1 obj1;  
obj1.showA1();  
obj1.showB1();
```

```
    C2 obj2;  
obj2.showA2();  
obj2.showB2();  
obj2.showC2();
```

```
    B3 obj3;    C3  
obj4;  
obj3.showA3();  
obj3.showB3();  
obj4.showA3();  
obj4.showC3();
```

```
    C4 obj5;  
obj5.showA4();  
obj5.showB4();  
obj5.showC4();    D5  
obj6;    obj6.showA5();  
obj6.showB5();  
obj6.showC5();  
obj6.showD5();
```

```
    return 0;  
}
```

```
Output
Single Inheritance A
Single Inheritance B
Multiple Inheritance A
Multiple Inheritance B
Multiple Inheritance C
Hierarchical Inheritance A
Hierarchical Inheritance B
Hierarchical Inheritance A
Hierarchical Inheritance C
Multilevel Inheritance A
Multilevel Inheritance B
Multilevel Inheritance C
Hybrid Inheritance A
Hybrid Inheritance B
Hybrid Inheritance C
Hybrid Inheritance D

=== Code Execution Successful ===
```

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

```
#include <iostream> using
namespace std;

class Base { public:  Base() {      cout <<
"Base Constructor Called" << endl;
}

~Base() {      cout << "Base Destructor
Called" << endl;
}
};

class Derived : public Base { public:  Derived() {
cout << "Derived Constructor Called" << endl;
}

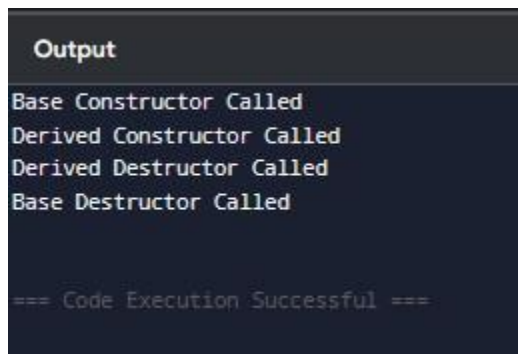
~Derived() {      cout << "Derived Destructor
Called" << endl;
}
}
```

```
};

int main() {

    Derived obj;

    return 0;
}
```



```
Output
Base Constructor Called
Derived Constructor Called
Derived Destructor Called
Base Destructor Called

=== Code Execution Successful ===
```

6. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```
#include <iostream> using
namespace std;

class Book { protected:
    string title;
    string author;
    float price;

public:
```

```
void setBook(string t, string a, float p) {  
title = t;    author = a;    price = p;  
}
```

```
void displayBook() {    cout <<  
"Title: " << title << endl;    cout <<  
"Author: " << author << endl;    cout <<  
"Price: " << price << endl;  
}  
};
```

```
class Textbook : public Book { private:  
    string subject;
```

```
public:  
    void setTextbook(string t, string a, float p, string s) {  
setBook(t, a, p);    subject = s;  
    }
```

```
    void displayTextbook() {  
        displayBook();    cout << "Subject:  
" << subject << endl;  
    }  
};
```

```
int main() {
```

```
    Book b;  
    b.setBook("The Alchemist", "Paulo Coelho", 399);  
cout << "General Book Details" << endl;  
b.displayBook();
```

```
    cout << endl;
```

```
    Textbook tb;    tb.setTextbook("Engineering Mathematics", "B.S. Grewal", 650,  
"Mathematics");    cout << "Textbook Details" << endl;    tb.displayTextbook();
```

```
    return 0;  
}
```

```
Output
General Book Details
Title: The Alchemist
Author: Paulo Coelho
Price: 399

Textbook Details
Title: Engineering Mathematics
Author: B.S. Grewal
Price: 650
Subject: Mathematics

=== Code Execution Successful ===
```

7. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

```
#include <iostream> using
namespace std;
```

```
class Speedometer {
protected:
    float speed;

public:
    void setSpeed(float s) {
        speed = s;
    }
}
```

```
    float getSpeed() {  
return speed;  
    }  
};
```

```
class FuelGauge {  
protected:  
    float fuelLevel;
```

```
public:  
    void setFuelLevel(float f) {  
fuelLevel = f;  
    }
```

```
    float getFuelLevel() {  
return fuelLevel;  
    }  
};
```

```
class Thermometer {  
protected:  
    float temperature;
```

```
public:  
    void setTemperature(float t) {  
temperature = t;  
    }
```

```
    float getTemperature() {  
return temperature;  
    }  
};
```

```
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer { public:  
    void displayDashboard() {        cout << "Speed: " << speed  
<< " km/h" << endl;        cout << "Fuel Level: " << fuelLevel  
<< " %" << endl;        cout << "Temperature: " <<  
temperature << " C" << endl;  
    }  
};
```

```

int main() {

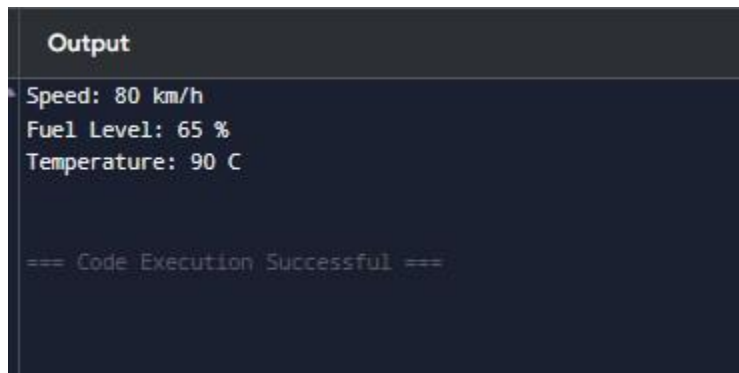
    CarDashboard car;

    car.setSpeed(80);
    car.setFuelLevel(65);
    car.setTemperature(90);

    car.displayDashboard();

    return 0;
}

```



The screenshot shows a dark-themed window titled "Output". Inside, the following text is displayed:

```

Speed: 80 km/h
Fuel Level: 65 %
Temperature: 90 C

=== Code Execution Successful ===

```

8. You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class `LibraryUser` with general attributes. Then, create two derived classes `Student` and `Teacher` that inherit from `LibraryUser` and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

```

#include <iostream> using
namespace std;

class LibraryUser {
protected:  string
name;  int id;
string contact;

```

public:

```
void setUser(string n, int i, string c) {  
    name = n;    id = i;    contact = c;  
}
```

```
void displayUser() {    cout <<  
"Name: " << name << endl;    cout <<  
"ID: " << id << endl;  
    cout << "Contact: " << contact << endl;  
}  
};
```

```
class Student : public LibraryUser { private:  
    string gradeLevel;
```

public:

```
void setStudent(string n, int i, string c, string g) {  
    setUser(n, i, c);    gradeLevel = g;  
}
```

```
void displayStudent() {    displayUser();  
cout << "Grade Level: " << gradeLevel << endl;  
}  
};
```

```
class Teacher : public LibraryUser { private:  
    string department;
```

public:

```
void setTeacher(string n, int i, string c, string d) {  
    setUser(n, i, c);    department = d;  
}
```

```
void displayTeacher() {    displayUser();  
cout << "Department: " << department << endl;  
}  
};
```

```
int main() {
```



```
Student s;  
s.setStudent("Saanvi", 1024, "7009756092", "2nd yr");  
cout << "Student Details" << endl; s.displayStudent();  
  
cout << endl;  
  
Teacher t;  
t.setTeacher("Mr. Sharma", 201, "9123456780", "Computer Science");  
cout << "Teacher Details" << endl; t.displayTeacher();  
  
return 0;  
}
```

```
Output  
Student Details  
Name: Saanvi  
ID: 1024  
Contact: 7009756092  
Grade Level: 2nd yr  
  
Teacher Details  
Name: Mr. Sharma  
ID: 201  
Contact: 9123456780  
Department: Computer Science  
  
=== Code Execution Successful ===
```

9. A logistics company needs a software system to manage its vehicle fleet. All vehicles share common attributes like make, model, and year. Trucks have additional attributes like load_capacity. Furthermore, refrigerated trucks have a special attribute called temperature_control. Implement a base class Vehicle with common attributes. Then, create a derived class Truck that adds the load_capacity attribute. Finally, create another derived class RefrigeratedTruck that inherits from Truck and adds the temperature_control attribute. Demonstrate how multilevel inheritance works in this case.

```
#include <iostream> using
namespace std;

class Vehicle { protected:
    string make;
    string model;    int
    year;

public:
    void setVehicle(string mk, string md, int y) {
make = mk;        model = md;        year = y;
    }

    void displayVehicle() {        cout <<
"Make: " << make << endl;        cout <<
"Model: " << model << endl;        cout <<
"Year: " << year << endl;
    }
};

class Truck : public Vehicle { protected:
    float load_capacity;

public:
    void setTruck(string mk, string md, int y, float lc) {
setVehicle(mk, md, y);
        load_capacity = lc;
    }

    void displayTruck() {        displayVehicle();        cout << "Load
Capacity: " << load_capacity << " tons" << endl;
```

```

    }
};

class RefrigeratedTruck : public Truck { private:
    float temperature_control;

public:
    void setRefrigeratedTruck(string mk, string md, int y, float lc, float temp) {
setTruck(mk, md, y, lc);    temperature_control = temp;
    }

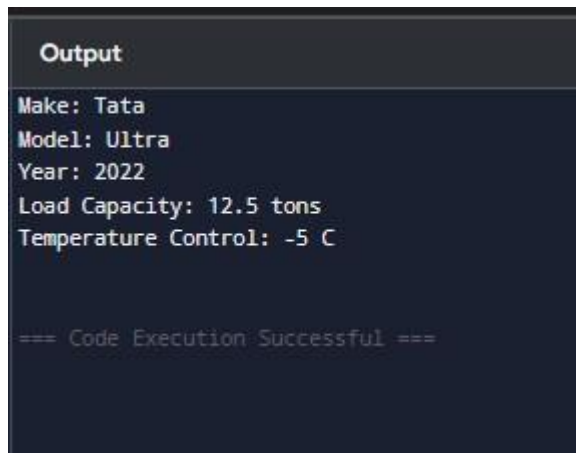
    void displayRefrigeratedTruck() {    displayTruck();    cout <<
"Temperature Control: " << temperature_control << " C" << endl;
    }
};

int main() {

    RefrigeratedTruck rt;    rt.setRefrigeratedTruck("Tata",
"Ultra", 2022, 12.5, -5);    rt.displayRefrigeratedTruck();

    return 0;
}

```



```

Output
Make: Tata
Model: Ultra
Year: 2022
Load Capacity: 12.5 tons
Temperature Control: -5 C

=== Code Execution Successful ===

```

You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student. A Person has general attributes like name and address. Staff members, who are a type of Person, have additional attributes like

employee_id and **department**. **Student**, another type of **Person**, has attributes like **student_id** and **grade**. Some **Staff** members are also **Students** (e.g., teaching assistants) and need to inherit from both classes. Implement a base class **Person** with general attributes. Then, create derived classes **Staff** and **Student** that inherit from **Person** and add their specific attributes. Finally, create a **TeachingAssistant** class that inherits from both **Staff** and **Student**. Demonstrate how hybrid inheritance is applied and managed in this scenario.

```
#include <iostream> using
namespace std;
```

```
class Person { protected:
    string name;
    string address;
```

```
public:
    void setPerson(string n, string a) {
name = n;      address = a;
    }

    void displayPerson() {      cout <<
"Name: " << name << endl;      cout <<
"Address: " << address << endl;
    }
};
```

```
class Staff : virtual public Person { protected:
    int employee_id;
    string department;
```

```
public:
    void setStaff(int id, string dept) {
employee_id = id;      department
= dept;
    }

    void displayStaff() {      cout << "Employee ID: "
<< employee_id << endl;      cout << "Department: "
<< department << endl;
    }
};
```

```

class Student : virtual public Person { protected:
    int student_id;
    string grade;

public:
    void setStudent(int id, string g) {
        student_id = id;    grade = g;
    }

    void displayStudent() {    cout << "Student
ID: " << student_id << endl;    cout << "Grade:
" << grade << endl;
    }
};

class TeachingAssistant : public Staff, public Student { public:
    void setTeachingAssistant(string n, string a, int eid, string dept, int sid, string g) {
        setPerson(n, a);
        setStaff(eid, dept);
        setStudent(sid, g);
    }

    void displayTeachingAssistant() {
        displayPerson();    displayStaff();
        displayStudent();
    }
};

int main() {

    TeachingAssistant ta;    ta.setTeachingAssistant("Saanvi", "Mansa", 501,
"Computer Science", 1201, "A");    ta.displayTeachingAssistant();

    return 0;
}

```

Output

Name: Saanvi
Address: Mks
Employee ID: 501
Department: ENC
Student ID: 1024
Grade: A

=== Code Execution Successful ===

